Ders 11 Genişletilmiş Notu: Soyut Sınıflar, Dinamik Bağlama, Kalıtımda Nesne Oluşturma Sırası ve Overload Çözümleme

Ders 11 Genişletilmiş Notu: Soyut Sınıflar, Dinamik Bağlama, Kalıtımda Nesne Oluşturma Sırası ve Overload Çözümleme

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Tekrar: Nesneler Arası İlişkiler
- 2. Tekrar: Overload ve Override
- 3. Soyut Metot ve Soyut Sınıf
- 4. Soyut Sınıfların Kuralları ve Kullanım Amacı
- 5. Dinamik Bağlama: PrintDriver Örneği
- 6. Çoklu Kalıtım ve Elmas Problemi
- 7. Tasarım Örneği: Kreş, Çocuk ve Eşya
- 8. Kalıtımda Nesne Oluşturma Sırası
- 9. Overload Çözümleme: Se7en Örneği
- 10. Uygulama: Kitaplık Sistemine Soyut Kart Sınıfı Eklemek

1. Tekrar: Nesneler Arası İlişkiler

Dr. Ahmet Elbir derse, kalıtım konusunun tamamlandığını hatırlatıp nesneler arasındaki ilişkileri sınıfa sorarak başladı.

Soru: Temelde kaç çeşit ilişkimiz vardı, bunlardan hangisi sınıf seviyesindeydi? **Cevap:** Beş çeşit ilişki vardır. Kalıtım (inheritance) sınıf seviyesinde, kalan dördü nesne seviyesindedir.

Hoca sınıf seviyesi ile nesne seviyesi arasındaki farkı şöyle açıkladı. Nesne fizikseldir, elle tutulup gözle görülür gibi düşünülür. Sınıf daha soyuttur. Kalıtım gerçeklenen bir şeyden çok kod tekrarını ve nesneye yönelik tasarımı ilgilendirir. Diğer ilişkiler ise oluşturulmuş fiziki nesnelerin birbiriyle bağını gösterir.

1.1 Association ve Dependency

Association ilişkisinde bir nesne, başka bir nesnenin referansını içinde saklar. Bunun için kullanılan anahtar kelime “has”tir. Derste görülen örnekler rafın içinde kitap saklanması, kişinin arabasının bilgisinin ya da arabanın sahibinin saklanmasıydı. Bu ilişkide nesne sayısı sıfır ya da çok olabilir. Bir kişinin hiç arabası olmayabilir, çok arabası da olabilir.

Dependency ilişkisinde ise bir nesne başka bir nesneyi yalnızca metotlarının içinde kullanır. Referans sınıfın alanı olarak saklanmaz. Gerektiğinde bir metodun içinde başka bir nesneye başvurulur. UML’de kesik çizgiyle gösterilir ve çizginin üstüne dereceler yazılabilir.

1.2 Aggregation ve Composition

Soru: Aggregation ve composition için "has" gibi dedik ama biraz daha fazlası var, burada ne diyorduk? **Cevap:** Parça-bütün (whole-part) ilişkisi diyorduk.

Hocaya göre parça-bütün ilişkisinin association'dan farkı, en az bir parça bulunmasıdır. Association'da nesne sayısı sıfır ya da çok olabilirken, parça-bütün ilişkisinde en az bir canlı nesneyle ilişki kurulmuş olmalıdır. Örnek otobüs hattıydı (bus road). Hattın üzerinde en az bir durak ya da n tane durak olur. Sıfır durak denirse bu association gibi olur.

> **Ders sonrası not:** Hoca parça-bütün ilişkisinde "en az bir parça" kuralını ayırt edici özellik olarak verdi. Bu, büyük olasılıkla dersin kullandığı bir yorumdur. UML'de bu ilişkilerde çokluk `0..*` de yazılabilir. Aggregation ile composition'ı ayıran asıl ölçüt, hocanın da anlattığı gibi parçanın yaşam süresidir.

Parça-bütün ilişkisi kendi içinde ikiye ayrılır. Her ikisi de düz çizgiyle çizilir, çizginin bir ucunda elmas (diamond) bulunur.

- **Aggregation (içi boş elmas):** Daha yumuşak, daha serbest bir ilişkidir. İçerideki küçük nesnenin referansı dışarıdan gönderilebilir. Hat nesnesi bir yerde, durak nesnesi başka bir yerde üretilir ve ikisi birbirinden bağımsız yaşayabilir. Hat silinince duraklar silinmiş olmaz.
- **Composition (içi dolu elmas):** Daha katı bir ilişkidir. Küçük nesne, büyük nesnenin içinde tanımlanır. Büyük nesne yok olunca içerideki nesne de yok olur. Hocanın verdiği genel örnek canlıdır. Canlının eli, kolu, yüzü vardır ve canlı yok olunca bunlar da gider.

Kısaca, birinde bütün yok olunca parça yaşamaya devam eder, ötekinde etmez.

1.3 Kalıtım: "is-a"

Soru: "Has" dedik, "whole-part" dedik, kalıtım için hangi anahtar kelimeyi söylemiştik? **Cevap:** "is-a".

B, A'nın özelliklerini alıyorsa "B is a A", yani "B bir A'dır" denir. UML'de üçgen uçlu okla gösterilir. Hoca bunu "A nesnesi, B nesnesi" diyerek anlattı, ama derste baştan belirtildiği gibi bu ilişki sınıf seviyesindedir. Örnekler: öğrenci bir kişidir, çalışan bir kişidir, emekli bir kişidir.

2. Tekrar: Overload ve Override

2.1 Overload

Soru: Overload neydi? **Cevap:** Bir metodu farklı parametre sayısıyla ya da farklı parametrelerle yazabilmekti.

Hoca bunu metodun imzasını değiştirmek olarak toparladı. Metodun adı aynı kalır, ad bölümünden sonrası, yani parametrelerin sayısı ya da türleri değişir. Hoca dönüş tipinin de aynı kalacağını söyledi.

> **Ders sonrası not:** Hoca overload'da dönüş tipinin aynı kalacağını söyledi; doğrusu, dönüş tipinin aynı kalması zorunlu değildir. Overload'u parametre listesinin farklı olması belirler. `static int f(int x)` ile `static double f(double x, int y)` birlikte derlenip çalıştı (javac ile denendi). Yalnız dönüş tipi farklı olan `int f(int)` ve `double f(int)` ise "method f(int) is already defined" hatası verdi (javac ile denendi).

2.2 Override

Soru: Override neydi? **Cevap:** Bir öğrenci "toString'i değiştiriyorduk" dedi. Hoca bunu kabul etmedi. Örnek üzerinden değil, bilgisayar mühendisi bakış açısıyla, terminolojik bir tanım istedi. Benzetmesi şuydu: üslü ve köklü sayılar sorulunca "karekök 25 eşittir 5'tir" demek gibi olur.

Soru: Üst sınıftaki bir metodu alt sınıfta parametresini değiştirerek ya da farklı bir şey ekleyerek yazmak diyebilir miyiz? **Cevap:** Parametresini değiştirerek değil. Parametre overload'da değişir, override'da değişmez. Devamında öğrenci, alt sınıfta `super` ile üst sınıftakini çağırıp eklenti yaptığımızı hatırlattı. Hoca `super` yazmanın zorunlu olmadığını ekledi.

Hocanın tanımı şudur: **Üst sınıftaki bir metodu alt sınıfta geçersiz kılmaya ya da ona eklenti yapmaya override denir.** Yani üst sınıftaki metot tamamen geçersiz kılınabilir ya da korunup üzerine eklenti yapılabilir.

Hoca bunu önceki derslerin `Person` sınıfındaki `bakiyeGuncelle` metoduyla gösterdi. Bakiye herkes için farklı güncellenir. Öğrencininki farklıdır, emeklininki farklıdır. Alt sınıftaki metot yukarıdakini geçersiz kılar. Yukarıdaki de çalışsın istenirse önce `super.bakiyeGuncelle()` çağrılır. Bu, üst sınıftaki metot soyut değilse mümkündür.

> **Ders sonrası not:** Derste ve uygulamada sık sık "overwrite" dendi; doğru terim override'dır. Hocanın ekranda yazdığı `@Override` işareti de bunu gösterir. Parametre listesi farklı bir metoda `@Override` yazılırsa "method does not override or implement a method from a supertype" hatası alınır (javac ile denendi).

3. Soyut Metot ve Soyut Sınıf

3.1 Sorunun Kurulması: Boş Metotlar

Hoca soyut sınıflara, uygulama saatinde yaptığı girişi tekrar ederek geçti. `Person` sınıfında `double bakiyeGuncelle()` metodu vardı ve içinde hiçbir şey yapılmadan `return 0.0` yazılmıştı. Çünkü herkesin bakiyesi bir şekilde güncellenecekti ama nasıl güncelleneceği alt sınıflarda belliydi. Hoca aynı durumu boş bir `public void yaz()` metodu ekleyerek gösterdi.

Önce metotların içi doluyken ne olduğuna baktı. `Person` içinde "Personel yaz", `Student` içinde "Öğrenci yaz" yazdıran metotlar vardı. `Student` nesnesi üretip `yaz()` çağrısı yapınca aşağıdaki metot çalışır, yukarıdaki çalışmaz. İkisi de çalışsın istenirse `Student` içindeki `yaz` metoduna `super.yaz()` eklenir:

```
class Person {
    public void yaz() { System.out.println("Personel yaz"); }
}

class Student extends Person {
    @Override
    public void yaz() {
        super.yaz();                          // önce üst sınıftaki çalışır
        System.out.println("Ogrenci yaz");
    }
}
```

`new Student().yaz()` çağrısının çıktısı önce `Personel yaz`, sonra `Ogrenci yaz` oldu (javac ile denendi).

Hoca burada sık yapılan bir karışıklığa dikkat çekti. `super.yaz()` metodu çağırır. Yalnız `super(...)` yazmak ise `yaz` metoduna gitmez, üst sınıfın constructor'ını çağırmak demektir ve constructor içinde kullanılır.

Sonra hoca asıl isteğini söyledi. `Person` içindeki `yaz` hiçbir şey yapmayacak, tamamen boş kalacak, ilgili alt sınıflarda doldurulacak. Böyle boş bırakılmış bir metot anlamsızdır. Ayrıca iki sorun vardır:

- Metodun yalnızca tanımlanacağı ifade edilmelidir, gövdesi olmamalıdır.
- `yaz` metodu sadece `Student` içinde yazıldı, `Teacher` içinde yazılması gerekmedi. Oysa istenen şudur: "Bu sınıftan kalıtım alıyorsan bu metodu tanımlamak zorundasın."

3.2 Çözüm: `abstract`

Bunun çözümü metodu soyut (abstract) tanımlamaktır. Gövde silinir, satır doğrudan noktalı virgülle biter.

```
public abstract void yaz();
public abstract double bakiyeGuncelle();
```

Soyut metoda gövde yazılırsa "abstract methods cannot have a body" hatası alınır (javac ile denendi).

Soyut metot yazılınca sınıf hata vermeye başlar. Buradaki kural hocanın üstünde en çok durduğu noktadır.

> **Hocanın vurgusu:** Bir sınıf en az bir soyut metot içeriyorsa soyut tanımlanmak zorundadır. Soyut tanımlanmış bir sınıfın ise soyut metot içermesi zorunlu değildir. Ters-düz ilişkisi kurmayın. Çoğu kişi "sınıf soyutsa soyut metot içerir" der, bu doğru bir tanım değildir.

Her iki yön de denendi. Soyut metot içeren ama `abstract` tanımlanmayan `Person` sınıfı "Person is not abstract and does not override abstract method yaz() in Person" hatası verdi. Hiç soyut metodu olmayan `abstract class Person` ise derlendi, alt sınıfıyla çalıştı (javac ile denendi).

3.3 Bir Kısıtlama, Bir Zorunluluk

Sınıf soyut olunca karşımıza bir kısıtlama ve bir zorunluluk çıkar. Hocaya göre bugünkü konu aslında budur.

Kısıtlama: Soyut sınıftan nesne üretilemez. Eskiden `Person` sınıfından nesne üretilebiliyordu, artık üretilemez. Yalnızca referans üretilebilir. Referansın türü `Person`, nesnenin kendisi `Student` olabilir. Bu, kalıtım konusunda görülmüştü. `new Person()` yazınca "Person is abstract; cannot be instantiated" hatası alınır (javac ile denendi).

Zorunluluk: `Person` sınıfından kalıtım alan sınıf, bütün soyut metotları gerçeklemek (implement etmek) zorundadır. Hoca IDE'de "implement" seçeneğini gösterdi. Bu seçenek soyut metotların hepsini alt sınıfa getirir ve hata gider, ama içini doldurmak programcıya kalır. Gerçeklemeyen sınıf derlenmez. `Student` yalnız `yaz` metodunu yazıp `bakiyeGuncelle` metodunu yazmayınca "Student is not abstract and does not override abstract method bakiyeGuncelle() in Person" hatası alındı (javac ile denendi).

Hocanın benzetmesi arabadır. Araba üreteceksen vitesini, gazını, frenini koymak zorundasın. Orada bir standart vardır. Soyut metot da böyle bir standart belirler. `Teacher` sınıfında da aynı durum geçerlidir.

Metot soyut olunca `Student` içindeki `super.yaz()` çağrısı da artık yazılamaz. Çünkü `yaz` üst sınıfta çalışabilir bir metot değildir. Hoca bunu C'deki fonksiyon prototipine benzetti: Orada prototip yazılır, burada da üstte prototip, altta gerçekleme yazılmış olur. javac bu çağrıda "abstract method yaz() in Person cannot be accessed directly" hatası verdi (javac ile denendi).

Bu bölümün derlenip çalıştırılan özeti şöyledir:

```
abstract class Person {
    private int id, yas;
    private double bakiye;
    public Person(int id, int yas) { this.id = id; this.yas = yas; }
    public void yasArttir() { yas++; }        // somut metot
    public int getYas() { return yas; }
    public abstract void yaz();               // soyut metot
    public abstract double bakiyeGuncelle();  // soyut metot
}

class Student extends Person {
    public Student(int id, int yas) { super(id, yas); }
    @Override public void yaz() { System.out.println("Ogrenci yaz"); }
    @Override public double bakiyeGuncelle() { return 0.0; }
}

public class T {
    public static void main(String[] a) {
        Person p = new Student(1, 20);   // referans Person, nesne Student
        p.yaz();                         // Ogrenci yaz
        p.yasArttir();
        System.out.println(p.getYas());  // 21
    }
}
```

Buradaki 1 ve 20 değerleri yalnız derleme denemesi içindir. Çıktı açıklamalarda yazıldığı gibi oldu (javac ile denendi).

4. Soyut Sınıfların Kuralları ve Kullanım Amacı

Hoca bu bölümü İngilizce slayt üzerinden anlattı.

4.1 Tanım ve Özellikler

Soyut sınıf, temel sınıf (base class) olarak kullanılan ve normal alt sınıfları için bir şablon (template) temsil eden sınıftır. Şablondur, fiziksel bir sınıf değildir. Şimdiye kadar yazdığımız normal sınıflara somut sınıf (concrete class) denir. Soyutun karşıtı somuttur.

- Soyut sınıf `abstract` anahtar kelimesiyle belirtilir.
- Soyut sınıftan örnek (instance) oluşturmak yasaktır. Örnek, soyut sınıfın somut alt sınıflarından oluşturulur. `Teacher`, `Student` ya da `Retired` sınıfından nesne üretilir, `Person` sınıfından üretilemez.
- Soyut sınıfların somut sınıflar gibi üye alanları olabilir. Bu açıdan bir fark yoktur.
- Hem somut hem soyut üye metotları olabilir. Slayttaki soyut `Person` sınıfında üye değişkenler, constructor'lar, set ve get metotları gibi somut metotlar ve iki soyut metot vardır.
- Soyut metodun yalnızca tanımı vardır ve `abstract` anahtar kelimesini taşır.
- Kalıtımla alınan soyut metotların gövdeleri somut alt sınıfta yazılmalıdır. Yazılmazsa derleme zamanı hatası (compile time error) alınır.

Hoca arayüzlerle (interface) kısa bir karşılaştırma yaptı. Arayüzlerde somut metot tanımlanamaz, ama bu yasak görelidir. Java'nın yeni sürümlerinde somut metot gibi çalışan `default` metotlar tanımlanabilir. Arayüzler ayrıca görülecek, soyut sınıflarda ise durum burada anlatıldığı gibidir.

> **Ek bilgi (derste anlatılmadı):** Arayüze `default` metot yazma olanağı büyük olasılıkla Java 8 ile gelmiştir. İçinde `default void selam()` bulunan bir arayüz bu derste kullanılan JDK 25 ile derlenip çalıştı (javac ile denendi).

4.2 Soyut Sınıflara Ne Zaman İhtiyaç Duyarız?

Kalıtım hiyerarşisinde yukarı çıktıkça sınıflar genelleşir. Bir noktada sınıflar o kadar genelleşir ki örneklenmelerine gerek kalmaz. Hoca örnek olarak `Object` sınıfını verdi. `Object` sınıfından nesne üretmeyiz, alt sınıflardan üretiriz.

İkinci örnek görsel programlamadan geldi. Görsel pencere oluşturmayı sağlayan `JFrame` sınıfı kalıtım yoluyla kullanılacaktır. Hiyerarşide alta inildikçe sınıflar daha çok işe yarar, çünkü doğrudan hedefe yönelik olan onlardır. Üsttekiler genel bir şablon gibi düşünülür.

Hocanın "güzel bir cümle" diye nitelediği slayt fikri şudur: Sınıflar arasında kalıtım ilişkisi kuruyoruz ve soyut sınıfla şunu garanti etmek istiyoruz: Özel sınıflar belirli metotları mutlaka barındırmak zorundadır. Araba örneğine dönersek, her arabanın gaz pedalı olacaktır. Tasarladığınız bir sınıftan başkaları kalıtım alacaksa, kesin olmasını istediğiniz metotları soyut tanımlarsınız.

4.3 UML'de Gösterim

UML sınıf şemasında soyut sınıf ve metot italik yazılarak ya da `<<abstract>>` stereotipi eklenerek gösterilir. İtalik bazen görünmeyebilir ve yanlış anlaşılabilir, bu yüzden stereotip eklemek yararlıdır. Stereotip, herhangi bir UML şemasında bir simge olağan anlamı dışında kullanıldığında başvurulan gösterimdir.

5. Dinamik Bağlama: PrintDriver Örneği

Slayttaki örnek `PrintDriver` sınıfıydı. Her yazıcı sürücüsünün bir `print` metodu vardır, ama her sürücü bunu farklı yapar. `PCL6Driver` bir yolla, başka bir sürücü başka bir yolla yazdırır. Bu yüzden `print` soyut tanımlanır, parametresi belgedir (`document`) ve gövdesi somut alt

sınıflarda yazılır. Hoca bunu bakiye güncelleme örneğine benzetti ve bakiye örneğinin günlük hayattan olduğu için iyi bir gösterim sağladığını söyledi. Bu işi `if` ile yapmanın tasarım açısından yarattığı sıkıntılar daha önce konuşulmuştu.

Üst sınıfta detaylar bilinmez, alt sınıflarda bütün detaylar yazılmak zorundadır. Burada **dinamik bağlama (dynamic binding)** devreye girer. Referans `PrintDriver` türünden olsun, karşısında `new PSDriver()` yazsın. Bu referans `PrintDriver` gibi görünür, ama gerçek nesne `PSDriver` nesnesidir ve onun `print` metodu çalışır. Sonra aynı referansa `new PCL6Driver()` atanırsa artık o nesnenin metodu çalışır. Referans hiç değişmez.

```
abstract class PrintDriver {
    public abstract void print(String document);
}

class PSDriver extends PrintDriver {
    @Override
    public void print(String document) {
        System.out.println("PSDriver: " + document);
    }
}

class PCL6Driver extends PrintDriver {
    @Override
    public void print(String document) {
        System.out.println("PCL6Driver: " + document);
    }
}

public class T {
    public static void main(String[] a) {
        PrintDriver p = new PSDriver();
        p.print("belge");              // PSDriver: belge
        p = new PCL6Driver();
        p.print("belge");              // PCL6Driver: belge
    }
}
```

Sürücü gövdeleri ve yazdırılan metinler derleme denemesi için kurulmuştur, slayttaki gövdeler altyazıda okunmuyor. Çıktı yorumlarda yazıldığı gibi oldu (javac ile denendi).

> **Ders sonrası not:** Hoca “derleyici hangi türden nesne üretilmişse gider onun metodunu çağırır” dedi; doğrusu, bu seçim çalışma zamanında gerçek nesnenin türüne göre yapılır. Yukarıdaki kodda iki çağrı da `PrintDriver` türündeki aynı `p` referansı üzerinden yazıldığı hâlde farklı metotlar çalıştı (javac ile denendi). Hoca aynı yerde “aynı nesne” dedi, sonra “aynı referans” diye düzeltti; değişmeyen referanstır, nesne değişir.

Bunun tasarıma katkısı şudur: Yeni bir yazıcı sürücüsü yazılacaksa bu sınıftan kalıtım alınır ve soyut metot gerçeklenir. Hoca soyut sınıflar için söylenecek şeylerin hemen hemen hepsinin bu olduğunu belirtti.

6. Çoklu Kalıtım ve Elmas Problemi

Bazen birden fazla üst sınıftan kalıtım gerekebilir. Asistan öğrenci hem öğrenci hem çalışandır. Bu senaryoya çoklu kalıtım (multiple inheritance) denir. İlk bakışta yeni sınıfı var olan iki sınıftan türetmek uygun görünür:

```
class AssistantStudent extends Student, Employee { }   // derlenmez
```

Bu yazım mümkün değildir, çünkü Java çoklu kalıtımı desteklemez. Bu satır derlenince `'{' expected` hatası verdi (javac ile denendi).

Slaytta normal tekli kalıtım hiyerarşisinin yanında çoklu kalıtım hiyerarşisi de gösterildi. Bu yapı ek bir karmaşıklık getirir: elmas problemi (diamond inheritance problem). `toString` meto-

dunu düşünelim. `Student` ve `Employee` bu metodu doğrudan override edebilir. İş `AssistantStudent` sınıfına gelince hangisinin kullanılacağı konusunda karışıklık çıkar.

> **Hocanın vurgusu:** Bu tür karmaşıklıklar yüzünden çoklu kalıtım Java ve C#'ta desteklenmez. Python'da ise çoklu kalıtım görülecek ve hoca bunun için örnek vereceğini söyledi.
>
> **Ek bilgi (derste anlatılmadı):** Java'daki yasak sınıflar içindir. Bir sınıf birden fazla arayüzü gerçekleyebilir. `class K implements D, E` yazımı derlenip çalıştı (javac ile denendi).

6.1 Araya Giren Soru: Somut Metotlar Boşa mı Gider?

Soru: `Person` sınıfındaki soyut olmayan metotlar bir daha kullanılmayacak, değil mi? **Cevap:** Hayır, bu yanlıştır. Hoca `yasArttir` metodunu gösterdi. Bu metodu override etme zorunluluğu yoktur. Override edilmezse üst sınıftaki kullanılır. Yaş güncelleme `Student`, `Teacher` ve emekli için aynıdır, herkesin yaşı birer birer artar. Bu yüzden bu metot hem burada hem alt sınıflarda kullanılır. `setYas` ve `setBakiye` gibi `public` metotlar da alt sınıflarda override edilebilir, ama zorunlu değildir.

Hoca ayrıca override etmenin üst metodu tamamen kullanılmaz hâle getirmek demek olmadığını tekrar etti. İçeride bir şey değiştirilebilir, eklenti yapılabilir, `super` ile hem yukarıdaki hem aşağıdaki çalıştırılabilir.

7. Tasarım Örneği: Kreş, Çocuk ve Eşya

Slayttaki UML şemasında `Item`, `Toy`, `Clothing`, `Child` ve `Kindergarten` sınıfları vardır. `Item` soyut sınıftır. `isSuitable` soyut metottur, diğer metotlar somuttur, ayrıca bir constructor vardır. `Toy` ve `Clothing` eşyanın özel türleridir ve `isSuitable` metodunun gerçeklemesi bu özel sınıflarda yazılır.

Senaryonun gerekçesi şudur: Her eşya her çocuğa uygun değildir. Oyuncakların bir alt yaş sınırı vardır ve bu genellikle yılla ölçülür. Giysilerin alt ve üst yaş sınırı vardır ve bu genellikle ayla ölçülür.

İlişkiler şöyle okunur:

- `Item` ile `Child` arasında dependency vardır. `Child`, bu metottan kullanılır.
- `Kindergarten` ile `Child` arasındaki ilişki, okun şekli nedeniyle association, belki aggregation'dır. Composition değildir, çünkü kreş ile çocuk arasında katı bir bağ yoktur.
- `Toy` ve `Clothing`, `Item` sınıfından kalıtım alır.

7.1 Sınıfları Yazma Sırası

Soru: Bu senaryoyu kodlayacak olsanız ilk hangi sınıfı yazarsınız? **Cevap:** Bir öğrenci `Item` dedi. Hoca bunun kabul edilebilir olduğunu, ama `Item` yerine önce `Child` ve `Kindergarten` tanımlanması gerektiğini söyledi. Çünkü `Item` içindeki metot `Child` türünü kullanır. Bu tür henüz yoksa, bu metot yazılırken böyle bir referans bulunamadı hatası alınır.

Sınıflar aslında istenen sırayla yazılabilir, ama sıra derleme hatalarını azaltmak açısından önemlidir. Hocanın önerdiği sıra şudur:

1. `Child`, dependency ile kullanıldığı için önce yazılır.
2. `Kindergarten` ya da `Item` eklenir.
3. En son alt sınıflar `Toy` ve `Clothing` yazılır.

> **Hocanın vurgusu:** Sınıfları gelişigüzel tanımlamaya kalkarsanız bir sürü derleme hatası alırsınız. Bu hataları düzeltmeye uğraşmak zaman kaybıdır.

7.2 `super` ile Üst Sınıf Bilgisini Göndermek

`Toy` nesnesi oluşturulurken `Item` sınıfına ait `barcode` ve `description` bilgileri constructor'a gelir. Bunlar `super` ile üst sınıfın constructor'ına gönderilmek zorundadır. Hoca genel durumu

şöyle ifade etti: Constructor'a dört bilgi geliyorsa ve ikisi üst sınıfa aitse, o ikisi `super` ile gönderilir, kalan ikisi burada atanır.

```
abstract class Item {
    private String barcode, description;
    public Item(String barcode, String description) {
        this.barcode = barcode;
        this.description = description;
    }
    public abstract boolean isSuitable(Child child);
}

class Toy extends Item {
    private int altYasYil;
    public Toy(String barcode, String description, int altYasYil) {
        super(barcode, description);   // üst sınıfa ait bilgiler
        this.altYasYil = altYasYil;    // bu sınıfa ait bilgi
    }
    // isSuitable burada yazılmak zorundadır
}
```

Bu parça anlatımdan kurulmuş bir taslaktır. `isSuitable` gerçeklemesi eklenince derlendi. `super(...)` çağrısı olmadan yazılan constructor ise "constructor Item in class Item cannot be applied to given types" hatası verdi (javac ile denendi).

> **Hocanın vurgusu:** Bu kalıp sınavda da soruldu. Üst sınıfa ait bilgilerin `super` ile gönderilmesi, kalıtımın kullanıldığını gösterir.

Hoca kalıtım konusunu burada bitirdi. Arayüzlere girmedi ve ek hazırladığı iki örneğe geçti.

8. Kalıtımda Nesne Oluşturma Sırası

8.1 Overload ve Polimorfizm Üzerine Kısa Bir Soru

Hoca örneğe geçmeden kavramları yokladı.

Soru: Overloading sadece kalıtımla ilgili bir kavram mıdır? **Cevap:** Hayır. Fonksiyon yazabildiğimiz her yerde, normal bir somut sınıfın içinde de yapılabilir. Overriding ise kalıtımla ilgilidir.

Soru: Overriding neye sebep oluyordu, öğrencide farklı, öğretmende farklı olmasıyla ilgili hangi kavramı söylemiştik? **Cevap:** Çok biçimlilik (polymorphism).

> **Hocanın vurgusu:** Bu kavramları adınız gibi bilmeniz lazım. Staj ya da iş başvurusunda polymorphism nedir, abstract nedir, inheritance nedir ve özellikleri nelerdir diye karşınıza çıkar. Hoca, proje yapacak öğrencilere de bu tür sorular sorduğunu ve cevap alamayınca "bu arkadaş yeteri kadar nesneye yönelik programlama bilmiyor" dediğini anlattı. Piyasada nesneye yönelik programlama bilmeyen birine çok yer olmadığını da ekledi.

8.2 Örnek Kod

Üç dereceli bir kalıtım vardır. En üstte `NoWait`, onun altında `Steve`, en altta `Alan` bulunur. Soru şudur: `main` çalışıp bir `Alan` nesnesi oluşturulunca arka planda neler olur? Hoca, kalıtım yapılınca arka planda ne olup ne olmayacağını bilmenin önemli olduğunu söyledi ve önce kodu konuşup sonra çalıştırdı.

```
class NoWait {
    public NoWait() { System.out.println("NoWait Konstruktor"); }
}

class Steve extends NoWait {
    static { System.out.println("Steve statischer Initialisierer"); }
    public Steve() { this(2); System.out.println("Steve Konstruktor 1"); }
```

```
    public Steve(int i) { super(); System.out.println("Steve Konstruktor 2"); }
}

public class Alan extends Steve {
    static NoWait alan = new NoWait();
    public Steve murmeltier = new Steve(5);
    public Alan() { System.out.println("Alan Konstruktor"); }
    public static void main(String[] args) { new Alan(); }
}
```

Kodun tam metni kısa nottan alındı, altyazıdaki adım adım anlatımla uyuşuyor.

8.3 "Nesne Oluşturulunca Ne Çalışır?"

Soru: Nesne oluşturulunca ne çalışır? **Cevap:** Öğrenciler constructor dedi. Hoca bunun laf arasında söylendiğini, ama doğru bir bilgi olmadığını belirtti.

Hoca bunu şu basit sınıfla gösterdi:

```
public class A {
    int x;
    public A(int a) {
        this.x = a;
    }
}
```

`A a = new A(...)` denince constructor çağrılır ve `this.x = a` işletilir. Ama bu satırın işletilebilmesi için önce sınıfın kendi `x` alanının oluşturulmuş olması gerekir. Yani önce alan oluşturulur, sonra constructor çalışır. Üye alan her zaman `int` olmaz, başka bir nesne de olabilir. O zaman işler karışır. Demek ki constructor'a her zaman "balıklama atlanmaz".

> **Ders sonrası not:** Hoca bu değişkenlere "lokal değişken" dedi; doğrusu üye alandır (member field). `x` değeri constructor'ın ilk satırında yazdırılınca 0, atamadan sonra 3 göründü. Alan, constructor gövdesine girilmeden önce oluşmuştur (javac ile denendi; 3 değeri yalnız bu deneme içindir).

Soru: Statik olan şey neye özeldi? **Cevap:** Sınıfa özeldi. Bir nesnenin olmasına gerek yoktur. Bu yüzden statikler nesneden bağımsız hareket eder.

8.4 Sıra Kuralı

> **Hocanın vurgusu:** Bu sırayı unutmayın. Kalıtım hiyerarşisi dikkate alınır. Önce statik değişkenler ve statik bloklar çalışır, en tepedekinin statiği önce gelir. Sonra sınıfa ait üye alanlar (member variables), en son constructor çalışır. Ondan sonra nesne üretilmiş olur. Ayrıca çalışmış bir statik bir daha çalışmaz.

Hoca izlemede şu ayrıntıyı da ekledi: Bir sınıfın üye alanları tanımlanmadan önce üst sınıfa gidilir ve üst sınıfların constructor'larına bakılır. Yani önce kalıtımla yukarıdaki işler halledilir, sonra alttaki sınıfın üye alanları oluşturulur, en son onun constructor gövdesi çalışır.

8.5 Adım Adım İzleme

Hoca kodu önce ekranda çıktı sırasını numaralandırarak, sonra çalıştırıp kod üzerinden izledi. İzleme şöyledir:

1. `Alan` sınıfına gelinir. Burada bir statik alan vardır, ama önce üst sınıfa bakılır. `Steve` sınıfına gidilir. `Steve` sınıfında statik blok vardır, ama ondan önce onun üstü olan `NoWait` sınıfına gidilir.
2. `NoWait` içinde statik bir şey yoktur. Hiyerarşide geri dönülür.
3. `Steve` sınıfının statik bloğu çalışır. Çıktı: `Steve statischer Initialisierer`.
4. `Alan` sınıfının statik alanı `alan = new NoWait()` değerlendirilir. `NoWait` sınıfının üstünde sınıf, statik değişken ve üye alan yoktur, doğrudan constructor çalışır. Çıktı: `NoWait Konstruktor`. Bu işlem bir defaya mahsustur.

5. Statikler bitti, sıra `new Alan()` nesnesindedir. `Alan()` boş constructor'dur, önce üst sınıfın boş constructor'ı olan `Steve()` çağrılır.
6. `Steve()` içindeki `this(2)` aynı sınıftaki `Steve(int)` constructor'ına gider. O da `super()` ile `NoWait()` constructor'ını çağırır. Çıktı: `NoWait Konstruktor`.
7. `Steve(int)` gövdesine dönülür. Çıktı: `Steve Konstruktor 2`.
8. `this(2)` bitti, `Steve()` gövdesine dönülür. Çıktı: `Steve Konstruktor 1`.
9. Üst sınıf kısmı bitti, sıra `Alan` sınıfının üye alanındadır: `murmeltier = new Steve(5)`. `Steve(int)` çağrılır, `i` değeri 5 olur. `super()` ile `NoWait()` çalışır. Çıktı: `NoWait Konstruktor`.
10. `Steve(int)` gövdesine dönülür. Çıktı: `Steve Konstruktor 2`. Statik blok yeniden çalışmaz.
11. Başka üretilecek nesne kalmadı, `Alan()` gövdesi çalışır. Çıktı: `Alan Konstruktor`. Nesne üretilmiş olur.

Program çıktısı şöyledir (javac ile denendi):

```
Steve statischer Initialisierer
NoWait Konstruktor
NoWait Konstruktor
Steve Konstruktor 2
Steve Konstruktor 1
NoWait Konstruktor
Steve Konstruktor 2
Alan Konstruktor
```

Hoca üye alanın sınıf içinde nereye yazıldığının fark etmediğini de gösterdi. `murmeltier` satırı başka bir yere yazılsa da önce o yapılır, constructor gövdesi ondan sonra çalışır. Satır constructor'ların altına taşınıp yeniden derlendi, çıktı birebir aynı kaldı (javac ile denendi).

> **Ders sonrası not:** Hoca `murmeltier` için "class variable referansı" dedi; doğrusu örnek değişkenidir (instance variable), çünkü `static` değildir. Bu yüzden her `Alan` nesnesi için oluşur, `alan` statik alanı ise bir kez oluşur. Hoca `new Steve(5)` için de "default olarak buna geleceğiz" dedi; çağrılan constructor boş constructor değil, `int` parametreli olandır. Sonuç, yukarıdaki çıktıda olduğu gibidir.

Hoca burada aslında bir kompozisyon işlemi olduğunu da belirtti, çünkü `Alan` içinde başka bir nesne üretilmektedir. Kalıtım ve başka bir nesnenin referansının oluşturulması işin içine girince sıra karmakarışık bir hâl alır. Hoca bu kodu paylaşacağını ve üzerinde çalışılabileceğini söyledi.

9. Overload Çözümleme: Se7en Örneği

Hoca, "birazcık kafanızı karıştıracağım" diye söz verdiği overload örneğine geçti. Aynı adlı dört metot vardır. Sorular şudur: Şu değişkenlerle çağrılınca hangisi çalışır, overload neye göre çalışır?

```
public class Se7en {
    public static void sin(int x, char y, int z)      { System.out.print("Bir "); }
    public static void sin(int x, short y, long z)    { System.out.print("İki "); }
    public static void sin(char x, float y, double z) { System.out.print("Üç "); }
    public static void sin(int x, int y, float z)     { System.out.print("Dört "); }

    public static void main(String[] args) {
        char c = '9'; short s = 8; int i = 7; long l = 6; float f = 5; double d = 4;
        sin(c, f, d);
        sin(i, c + s, i);
        sin(i, c, i);
        sin(i, i, f);
        sin(s + s, s, s);
        sin(s, s, s);
    }
}
```

Bu kodun çıktısı `Üç Dört Bir Dört İki İki` oldu (javac ile denendi).

9.1 Birebir Eşleşen Çağrılar

`sin(c, f, d)` çağrısında hoca parametreleri sırayla karşılaştırdı. `c` karakterdir, karakter alan bir metot vardır. `f` float, `d` double'dır. Metot doğrudan uyar, "Üç" çalışır. Birebir eşleşmede bir sıkıntı yoktur.

`sin(i, c, i)` çağrısı `int, char, int` ile birebir eşleşir, doğrudan "Bir" çalışır.

`sin(i, i, f)` çağrısı `int, int, float` ile birebir eşleşir, "Dört" çalışır.

9.2 `sin(i, c + s, i)` ve Hocanın Şaşırdığı Yer

Hoca ikinci çağrıda karakterle `short` toplanınca ne olacağını sordu ve bu toplamın `short` olacağını düşünerek ilerledi. Üçüncü argüman `int` olduğu için `long` isteyen "İki" metoduna gitmeye gerek kalmayacağını söyledi, toplamın karaktere dönüştürülebileceğini de düşündü. "Üç" ve "Dört" metotlarının çalışmayacağını söyleyip eledi. Sonra ekranda "Dört" görününce "4 mü, 4 nasıl çalıştı?" diye şaşırdı. Tekrar çalıştırdı, sonuç değişmedi. Bir ara "karakterle short'u toplayınca integer yapar" dedi, sonra "short yapar" diye geri döndü. Açıklaması da şu oldu: `short` karakterden büyüktür, karaktere dönüştürülemez, bu yüzden en uygun olarak `(int, int, float)` seçildi.

> **Ders sonrası not:** Hoca "karakterle short toplanınca short olur" dedi; doğrusu, işlemin sonucu `int`'tir. Java'da `char + short` işleminde iki işlenen de `int`'e yükseltilir. `short t = c + s;` satırı "possible lossy conversion from int to short" hatası verdi (javac ile denendi). Hocanın gerekçesi doğru olsaydı sonuç da farklı çıkardı: İkinci argümanı gerçekten `short` olan `sin(i, s, i)` çağrısı "İki" yazdırdı (javac ile denendi). "Dört" sonucunun asıl nedeni, `int` türündeki ikinci argümanın `char` ya da `short` parametreye daralmamasıdır. Geriye tek uyan metot olarak `(int, int, float)` kalır, üçüncü argüman da `int`'ten `float`'a genişler.
>
> **Ders sonrası not:** Hoca "short'un ASCII kodunu topluyorsunuz" dedi; doğrusu, karakterin sayısal değeri ile `short` değerin kendisi toplanır. Bu koddaki değerlerle `'9'` karakterinin kodu 57'dir, `c + s = 57 + 8 = 65` olur (ara adım eklendi, python ile hesaplandı). Hoca `short` için "karakterden büyük" dedi; iki tip de 16 bittir (`Short.SIZE` ve `Character.SIZE` java ile yazdırıldı) ve hiçbiri diğerine otomatik dönüşmez. `short t = c;` ve `char u = s;` satırlarının ikisi de kayıplı dönüşüm hatası verdi (javac ile denendi).

9.3 Küçükten Büyüğe Dönüşen Çağrılar

Kısa nottaki çağrı listesinde kalan ikisi aynı mantıkla açıklanır. Her ikisinde de ilk argüman `int`'e uyabilir, `short` da `char` parametreye gidemez:

- `sin(s + s, s, s)`: `s + s` işleminin sonucu `int`'tir. Argümanlar `int, short, short` olur. Sonuç "İki" (javac ile denendi).
- `sin(s, s, s)`: Argümanlar `short, short, short` olur. `short` küçükten büyüğe `int` ve `long` tiplerine genişler. Sonuç "İki" (javac ile denendi).

Bu iki çağrıda `(int, int, float)` metodu da uyar, ama derleyici `(int, short, long)` metodunu seçer. Hoca bu seçimi "en yakın olan, en uygun olan" diye ifade etti.

> **Ek bilgi (derste anlatılmadı):** Birden fazla metot uyduğunda derleyici en özel metodu seçer. Bir metodun bütün parametreleri diğerinin parametrelerine genişleyebiliyorsa o metot daha özeldir. `short` `int`'e, `long` da `float`'a genişleyebildiği için `(int, short, long)`, `(int, int, float)` metodundan daha özeldir. Bu seçim `sin(s, s, s)` çağrısında, hiçbiri diğerinden özel olmayınca verilen belirsizlik hatası da `sin(c, i, i)` çağrısında görüldü (javac ile denendi).

9.4 Büyükten Küçüğe Dönüşüm: Hata

Hoca sonra `sin(i, l, s)` çağrısını açtı ve derleme hatasını okudu. Bir metot uymaz, çünkü `long` değeri `char` parametreye kayıplı dönüşür. Kayıplı dönüşüme izin verilmez. `(int, short, long)` da uygulanabilir değildir. Aynı çağrının javac çıktısı şöyledir (javac ile denendi):

```
error: no suitable method found for sin(int,long,short)
  method sin(int,char,int) is not applicable
    (argument mismatch; possible lossy conversion from long to char)
  method sin(int,short,long) is not applicable
    (argument mismatch; possible lossy conversion from long to short)
  method sin(char,float,double) is not applicable
    (argument mismatch; possible lossy conversion from int to char)
  method sin(int,int,float) is not applicable
    (argument mismatch; possible lossy conversion from long to int)
```

> **Hocanın vurgusu:** Küçük tipten büyük tipe dönüşümde problem yoktur. Büyük tipten küçük tipe dönüşümde hata alınır.
>
> **Ders sonrası not:** Hoca açıp kapattığı hatalar için bir yerde "küçükten büyüğe dönüşümlerden kaynaklı hatalar" dedi; doğrusu, hemen önce kendisinin de söylediği gibi bu hatalar büyükten küçüğe dönüşümden kaynaklanır.

9.5 Belirsizlik: `sin(c, i, i)`

Hoca aradığı son durumu buldu: "ambiguous" hatası. Çağrı `sin(c, i, i)` çağrısıdır, argümanlar `char, int, int` türündedir. Metotları sırayla eledi:

1. `(int, char, int)`: İlk argüman `char`'dan `int`'e dönüşür, ama ikinci argüman `int`'ten `char`'a dönüşemez. Uymaz.
2. `(int, short, long)`: İkinci argüman `int`'ten `short`'a dönüşemez. Uymaz.
3. `(char, float, double)`: Karakter uyar, `int` `float`'a dönüşür, `int` `double`'a dönüşür. Uyar.
4. `(int, int, float)`: Karakter `int`'e dönüşür, `int` zaten `int`'tir, `int` `float`'a dönüşür. Uyar.

İki metot da uyduğu için derleyici "hangisini çalıştıracağımı bilmiyorum" der. Hoca bunun daha önce görülen uygulanamaz hatalarından farklı bir hata olduğunu vurguladı. javac çıktısı şöyledir (javac ile denendi):

```
error: reference to sin is ambiguous
  both method sin(char,float,double) in T and method sin(int,int,float) in T match
```

10. Uygulama: Kitaplık Sistemine Soyut Kart Sınıfı Eklemek

Dersin ikinci bölümünde Toprak Hoca, bir önceki uygulamadaki kitaplık örneği üzerinden geçen haftaki ve bu haftaki konuları tek bir senaryoda birleştirdi.

10.1 Önceki Kodda Yapılan Değişiklikler

İlk değişiklik overload içindir. Kitaplık ve raf sınıflarında farklı adlı kitap silme metotları vardı. IDE'nin "refactor" seçeneğiyle bunlar aynı ada, `kitapSil` adına getirildi. Toprak Hoca bunun overload olduğunu söyledi ve hangi metodun çalışacağının nasıl bilindiğini açıkladı: gönderilen parametrelere göre. Overload, farklı parametrelerle aynı metot adı demektir.

`Kitap` sınıfına `private double fiyat` alanı eklendi. Constructor'da ad, yazar ve üçüncü bilgi zaten alınıyordu. Fiyat constructor'dan tek tek verilmesin diye sabit olarak atandı, kitaplar 10 lira oldu. Toprak Hoca burada `this` yazılabileceğini, parametreyle ad çakışması olmadığı için yazılmayabileceğini de belirtti. Fiyata erişmek için bir get metodu eklendi.

`kitapAra` metodu da değiştirildi. Bulunan kitabı döndürmek yerine kitabın bulunduğu rafın numarasını döndürür, bulamazsa `-1` döndürür.

10.2 UML'deki Oklar

Yeni tasarımda soyut bir `Kart` sınıfı, ondan türeyen iki kart ve satışı yöneten bir `KitaplikYonetim` sınıfı vardır. Kart, içine para yüklenip onunla alışveriş yapılan bir karttır. Toprak Hoca şemadaki okları şöyle okudu:

- **Elmas:** Bir sınıfın içinde ötekinin dizisi vardır. Kitaplıkta raf dizisi, rafta kitap dizisi gibi.

- **Normal ok:** Sınıfın üye alanları arasında ötekinden bir değişken vardır. `KitaplikYonetim` içinde `kitaplik` değişkeni gibi.
- **Kesik (tırtıklı) ok:** Sınıfın alanlarında bulunmaz, ama bir metodunda kullanılır. `KitaplikYonetim` ile `Kart` arasındaki bağ böyledir.
- **Kalıtım okları:** Alt kart sınıflarının `Kart` sınıfından kalıtım aldığını gösterir.

Yapılan ödeme işi aslında çok basittir, ama sorumluluklar ayrı yerlerde yönetildiği için modüler bir yapı sağlanır.

10.3 Soyut `Kart` Sınıfı

`Kart` sınıfının `id`, `sahibi` ve `bakiye` alanları, bir constructor'ı, get ve set metotları ile `toString` metodu vardır. Ödeme metodunun yalnızca imzası yazılır. Bu metot `boolean` döndürür ve fiyat alır.

```
public abstract class Kart {
    private int id;
    private String sahibi;
    private double bakiye;

    public Kart(int id, String sahibi, double bakiye) {
        this.id = id;
        this.sahibi = sahibi;
        this.bakiye = bakiye;
    }

    public abstract boolean odemeYap(double fiyat);

    public String getSahibi() { return sahibi; }
    public double getBakiye() { return bakiye; }
    public void setBakiye(double bakiye) { this.bakiye = bakiye; }

    @Override
    public String toString() {
        return "Kart{id=" + id + ", sahibi=" + sahibi + ", bakiye=" + bakiye + "}";
    }
}
```

Bu sınıftan doğrudan nesne oluşturulamaz. Tek başına anlamlı olması için ondan kalıtım alınması gerekir. Bir tane soyut metot varsa sınıfın da soyut olması gerektiği burada tekrar görüldü.

10.4 `YetiskinKart`

```
public class YetiskinKart extends Kart {
    public YetiskinKart(int id, String sahibi, double bakiye) {
        super(id, sahibi, bakiye);
    }

    @Override
    public boolean odemeYap(double fiyat) {
        if (getBakiye() >= fiyat) {
            setBakiye(getBakiye() - fiyat);
            return true;
        }
        return false;
    }
}
```

`super(...)`, üstteki constructor'ı çağırır. Bakiye fiyata yetiyorsa bakiye azaltılır ve `true` döner, yetmiyorsa `false` döner. Toprak Hoca bunu, kartlarda görülen "başarılı" yazısına benzetti.

Bu sınıfta `getBakiye` ve `setBakiye` yazılmadı, `extends Kart` ile alındı. Toprak Hoca kalıtımı "üst sınıftaki her şeyin buraya kopyalanıp yapıştırıldığını düşünün" diye anlattı.

> **Ders sonrası not:** Toprak Hoca üst sınıftaki alanlara da "erişebiliriz" dedi; doğrusu, `private` alanlara alt sınıftan doğrudan erişilemez. Alt sınıfta `return bakiye;` yazınca "bakiye has private access in Kart" hatası alındı (javac ile denendi). Bu yüzden kodda `getBakiye` ve `setBakiye` kullanılır.

10.5 `OgrenciKart`

Öğrenci kartında bir farklılık olmalıdır, yoksa aynı sınıf kullanılabilirdi. Fark `bonus` alanıdır. Harcama yaptıkça bonus kazanılır ve bu, `odemeYap` metodunun davranışını değiştirir. Kart ilk oluştuğunda bonus 0'dır. Üst sınıfta soyut tanımlandığı için `odemeYap` bu sınıfta da yazılmak zorundadır.

```
public class OgrenciKart extends Kart {
    private double bonus = 0;

    public OgrenciKart(int id, String sahibi, double bakiye) {
        super(id, sahibi, bakiye);
    }

    public double getBonus() { return bonus; }
    public void setBonus(double bonus) { this.bonus = bonus; }

    @Override
    public boolean odemeYap(double fiyat) {
        if (getBakiye() >= fiyat) {
            setBakiye(getBakiye() - fiyat);
            bonus += fiyat * 0.2;     // setBonus(getBonus() + fiyat * 0.2) ile aynı
            return true;
        } else if (getBonus() >= fiyat) {
            setBonus(getBonus() - fiyat);
            return true;
        }
        return false;
    }

    @Override
    public String toString() {
        return super.toString() + " bonus=" + bonus;
    }
}
```

Bakiye yetiyorsa ödeme yapılır ve ödenen fiyatın %20'si bonusa eklenir. Yetmiyorsa bonusun yetip yetmediğine bakılır, yetiyorsa fiyat bonustan düşülür. Toprak Hoca aynı sınıfta olunduğu için `bonus` alanına doğrudan erişilebildiğini, `setBonus` ve `getBonus` ile yazmanın da aynı işi yaptığını gösterdi.

10.6 İlk Testler ve `toString` Override'ı

Önce yetişkin kartı denendi: `new YetiskinKart(0, "Ali", 50)`. `Ali` burada kart sahibi için örnek değerdir.

- **20 TL ödeme:** Verilen bakiye 50 TL'dir. $50 - 20 = 30$ TL kalır (python ile hesaplandı, java ile çalıştırıldı).
- **60 TL ödeme:** $50 < 60$ olduğu için ödeme yapılamaz, `false` döner, bakiye 50 TL kalır. Ödenemedi diye bir şey yazdırılmadığı için ekranda yalnızca bakiyenin değişmediği görülür.

Öğrenci kartı yazdırılınca bonus görünmedi. Çünkü `OgrenciKart` içinde `toString` yoktu ve `Kart` sınıfındaki `toString` çalışıyordu. O metodun bonustan haberi yoktur. Toprak Hoca `OgrenciKart`

için IDE ile yeni bir `toString` ekledi. Bu kez de yalnız bonus yazıldı. Üstteki bilgileri de yazdırmak için get metotları kullanılabilirdi, ama `super` kullanımını göstermek için `super.toString()` çağırıp üstüne bonus bilgisini ekledi.

Öğrenci kartıyla deneme:

- **60 TL ödeme:** Bakiye 50 TL, bonus 0 TL'dir, ödeme yapılamaz.
- **30 TL ödeme:** $50 - 30 = 20$ TL bakiye kalır, bonus $30 \times 0{,}2 = 6$ TL olur (python ile hesaplandı, java ile çalıştırıldı).

İlk denemede bakiye azalmadı, çünkü ödeme yanlışlıkla `kart1` üzerinden yapılmıştı. Düzeltilince değerler yukarıdaki gibi oldu.

Toprak Hoca override'ı burada şöyle tanımladı: Kalıtımla üst sınıftan alınan metodu alt sınıfta tekrar yazmak. Bu örnekte `toString`, `Kart` sınıfından alınıp `OgrenciKart` sınıfında yeniden yazıldı.

10.7 `KitaplikYonetim` ve Kitap Satışı

Satış sorumluluğu ayrı bir sınıfa verildi. Bu sınıfın `Kitaplik` tipinde bir üye alanı vardır, değeri constructor'da verilir.

```
public class KitaplikYonetim {
    private Kitaplik kitaplik;

    public KitaplikYonetim(Kitaplik kitaplik) {
        this.kitaplik = kitaplik;
    }

    public boolean kitapSat(Kitap kitap, Kart kart) {
        int rafNo = kitaplik.kitapAra(kitap.getAd());
        if (rafNo == -1) {
            return false;                    // kitap yok, satılamaz
        } else if (kart.odemeYap(kitap.getFiyat())) {
            System.out.println("Satış gerçekleşti");
            return true;
        } else {
            System.out.println("Yetersiz bakiye");
            return false;
        }
    }
}
```

Metot sırayla şunları yapar:

1. Olmayan kitap satılamaz. `kitapAra` kitabın adıyla çağrılır, çünkü bu metot adla arama yapar.
2. Sonuç `-1` ise kitap yoktur, `false` döner.
3. Kitap varsa `kart.odemeYap(kitap.getFiyat())` çağrılır. Ödeme yapıldıysa `true` geldiği için `if` içine girilir, "satış gerçekleşti" yazılır ve `true` döner.
4. Ödeme `false` dönerse bakiye yoktur, "yetersiz bakiye" yazılır.

Kitaplıkta aynı kitap birden çok kez eklendi, kitaplar 10 liradır ve iki kartın da 50 lirası vardır. Kitap 6 kez satılmaya çalışıldı.

Yetişkin kartı: Her satışta bakiye 10 azalır. 5 satıştan sonra $50 - 5 \times 10 = 0$ TL kalır. 6. denemede "yetersiz bakiye" yazılır, yalnız 5 kitap satılır.

Öğrenci kartı: Her ödeme 2 TL bonus kazandırır, çünkü $10 \times 0{,}2 = 2$. Adımlar şöyledir (ara adım eklendi):

Satış	Ödeme kaynağı	Bakiye (TL)	Bonus (TL)
Başlangıç		50	0
1	Bakiye	40	2

Satış	Ödeme kaynağı	Bakiye (TL)	Bonus (TL)
2	Bakiye	30	4
3	Bakiye	20	6
4	Bakiye	10	8
5	Bakiye	0	10
6	Bonus	0	0

Python ile hesaplandı ve yukarıdaki sınıflar java ile çalıştırıldı, tablo aynı çıktı. Bu çalıştırmada önceki uygulamadan gelen `Kitap` ve `Kitaplik` sınıflarının yalnız gereken kısmı yazıldı, çünkü bu sınıfların tamamı derste yeniden yazılmadı. Öğrenci kartı bu sayede 6. kitabı da aldı, yetişkin kartı alamadı.

10.8 Polimorfizm

Toprak Hoca kartları bu kez `Kart` türünde referansla oluşturdu:

```
Kart kart1 = new YetiskinKart(0, "Ali", 50);
```

Davranış değişmez, çünkü yetişkin kartı da sonuçta bir karttır ve `kitapSat` metodu zaten `Kart` türünde parametre bekler. Bunun faydası şudur: Bir gün başka özellikleri olan yeni bir kart kalıtımla oluşturulursa `KitaplikYonetim` ve diğer sınıflar hiç değişmeden o kartla ödeme yapabilir. Yeter ki kart `odemeYap` metodunu gerçeklesin ve `true` ya da `false` döndürsün. Böylece kartları geliştiren ekip ile satışı geliştiren ekip birbirinin koduna bağımlı olmadan çalışabilir, biri değişiklik yapınca "sen de şurayı değiştir" demek zorunda kalmaz.

Toprak Hoca iki tasarım kararını da özetledi. `Kart` sınıfını soyut yapınca ondan nesne oluşturulamaz oldu, sınıfın anlamlı olması için ondan kalıtım almak gerekti. `odemeYap` metodunu soyut yapınca da kalıtım alanlar onu yazmak zorunda kaldı. Diğer metotlarda böyle bir zorunluluk yoktur. `getSahibi` tekrar yazılmadı, `toString` ise öğrenci kartında tekrar yazıldı. İstenirse yazılır, istenmezse yazılmaz.

10.9 Kalıtımın Anlamı

Toprak Hoca kalıtımı günlük dile de uyan bir biçimde özetledi. Bunların hepsi birer karttır. Üst sınıfın bütün üye alanları ve fonksiyonları, kalıtım alan sınıfta da var kabul edilir. Üstüne fazladan alan ve metot eklenebilir. Bunu bir "büyük eşit" ilişkisi gibi düşünebilirsiniz: Kalıtım alan sınıf her zaman en az üst sınıf kadar, genelde daha fazla şey yapabilir. Daha özelleşmiş hâlidir.

Dersin Toparlaması

Bu derste nesneler arası ilişkiler ve overload-override tekrarından yola çıkılarak soyut sınıf konusu tamamlandı. Soyut metot bir standart koyar. Soyut metot içeren sınıf soyut olmak zorundadır, tersi geçerli değildir. Soyut sınıftan nesne üretilmez ama referans üretilir, somut alt sınıflar da bütün soyut metotları yazmak zorundadır. Referans türü aynı kalsa bile çalışan metodu gerçek nesnenin türü belirler, bu dinamik bağlamadır. Override ise çok biçimliliğe yol açar. Java'da sınıflar arasında çoklu kalıtım elmas problemi nedeniyle yoktur. Ardından iki ayrıntı incelendi: Nesne oluşurken sıra statikler, üye alanlar, constructor'dır ve hiyerarşi yukarıdan aşağı izlenir. Overload çözümlemede küçükten büyüğe dönüşüm serbest, büyükten küçüğe dönüşüm yasaktır. Birden çok metot eşit derecede uyarsa derleyici belirsizlik hatası verir. Uygulama bölümünde bunların hepsi kart ve kitap satışı senaryosunda birleşti.

Arayüzlere (interface) bu derste girilmedi, hoca 122. slayttan devam edileceğini söyledi. Görsel programlamada `JFrame` sınıfının kalıtımla kullanılacağını da belirtti. Dersin başında ayrıca, Veri Yapıları dersinde görülen ağaçların Java'daki kullanımının bu derste gösterilebileceğini söylemişti: Algoritmik taraf orada, kullanım tarafı burada konuşulacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.